

LAPORAN PRAKTIKUM
ALGORITMA PEMROGRAMAN 2
MODUL 5 - REKURSIF



NAMA : SAMINI

NIM : 109082500119

KELAS S1IF-13-06

PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO
2026

A. Dasar Teori

Rekursif merupakan teknik dalam pemrograman dimana suatu subprogram (fungsi atau prosedur) dapat memanggilnya sendiri untuk menyelesaikan suatu permasalahan. Konsep dasar dari rekursif adalah memecah masalah utama menjadi sub-masalah yang lebih kecil namun memiliki bentuk yang sama. Penyelesaian dilakukan secara berulang hingga mencapai kondisi tertentu. Contohnya adalah fungsi yang terus mencetak nilai dengan memanggilnya sendiri, sehingga tanpa kontrol, proses tersebut dapat berjalan tanpa henti.

Rekursif membutuhkan dua komponen utama, yaitu **base-case** dan **recursive-case**. Base-case adalah kondisi berhenti yang berfungsi untuk menghentikan proses rekursif agar tidak berjalan terus-menerus, sedangkan recursive-case adalah bagian dimana fungsi memanggil dirinya sendiri. Tanpa adanya kasus dasar, program akan mengalami rekursi tak terbatas atau perulangan tanpa batas. Penentuan kasus dasar menjadi hal paling penting dalam perancangan algoritma rekursif.

Proses rekursif terdiri dari dua tahap yaitu maju dan mundur. Pada tahap maju, fungsi akan terus memanggil dirinya hingga kondisi base-case terpenuhi. Setelah itu, proses dilanjutkan ke tahap backward, dimana eksekusi kembali ke pemanggilan sebelumnya secara berurutan. Tahap ini memungkinkan hasil diproses saat fungsi kembali (return), sehingga urutan output bisa berbeda tergantung posisi perintah dalam fungsi. Teknik rekursif ini juga dapat menjadi alternatif dari perulangan (iteratif) dalam menyelesaikan masalah tertentu.

B. Guided

1. [n hingga 1]

```
package main
import "fmt"
func main(){
    var n int
    fmt.Scan(&n)
    baris(n)
}

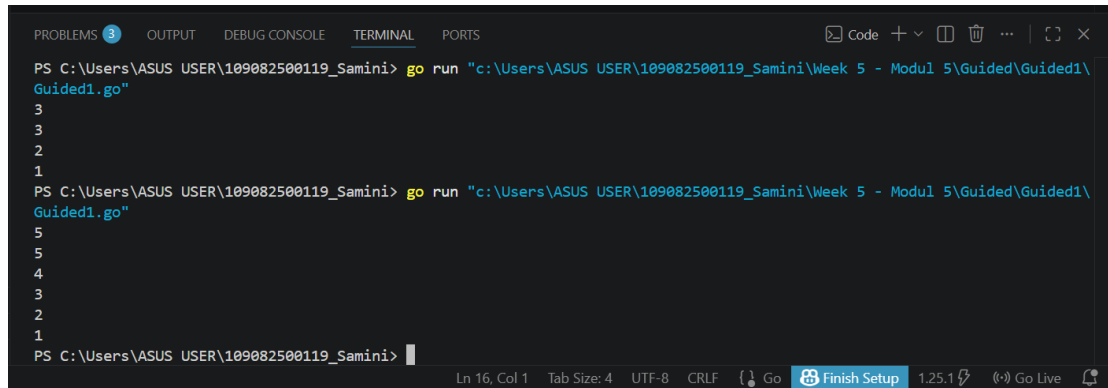
func baris(bilangan int){
    if bilangan == 1 {
        fmt.Println(1)
```

```

    }else{
        fmt.Println(bilangan)
        baris(bilangan - 1)
    }
}

```

Output :



```

PS C:\Users\ASUS USER\109082500119_Samini> go run "c:\Users\ASUS USER\109082500119_Samini\Week 5 - Modul 5\Guided\Guided1\Guided1.go"
3
3
2
1
PS C:\Users\ASUS USER\109082500119_Samini> go run "c:\Users\ASUS USER\109082500119_Samini\Week 5 - Modul 5\Guided\Guided1\Guided1.go"
5
5
4
3
2
1
PS C:\Users\ASUS USER\109082500119_Samini>

```

2. [1 hingga n]

```

package main
import "fmt"
func main(){
    var n int
    fmt.Scan(&n)
    fmt.Println(penjumlahan(n))
}
func penjumlahan(n int) int {
    if n == 1 {
        return 1
    }else{
        return n + penjumlahan(n-1)
    }
}

```

Output :

```
PROBLEMS 3 OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\ASUS USER\109082500119_Samini> go run "c:\Users\ASUS USER\109082500119_Samini\Week 5 - Modul 5\Guided\Guided2\Guided2.go"
5
15
PS C:\Users\ASUS USER\109082500119_Samini> go run "c:\Users\ASUS USER\109082500119_Samini\Week 5 - Modul 5\Guided\Guided2\Guided2.go"
3
6
PS C:\Users\ASUS USER\109082500119_Samini> go run "c:\Users\ASUS USER\109082500119_Samini\Week 5 - Modul 5\Guided\Guided2\Guided2.go"
6
21
PS C:\Users\ASUS USER\109082500119_Samini>
```

3. [Mencari dua pangkat n atau 2]

```
package main
import "fmt"
func main(){
    var n int
    fmt.Scan(&n)
    fmt.Println(pangkat(n))
}
func pangkat(n int) int {
    if n == 0 {
        return 1
    }else{
        return 2 * pangkat(n-1)
    }
}
```

Output :

```
PROBLEMS 3 OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\ASUS USER\109082500119_Samini> go run "c:\Users\ASUS USER\109082500119_Samini\Week 5 - Modul 5\Guided\Guided3\Guided3.go"
5
32
PS C:\Users\ASUS USER\109082500119_Samini> go run "c:\Users\ASUS USER\109082500119_Samini\Week 5 - Modul 5\Guided\Guided3\Guided3.go"
7
128
PS C:\Users\ASUS USER\109082500119_Samini>
```

C. Unguided

1. Soal Unguided 1

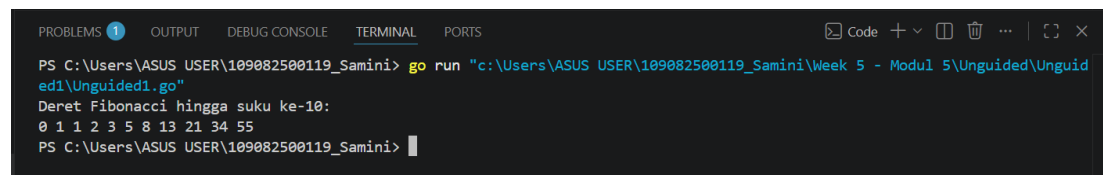
Deret fibonacci adalah sebuah deret dengan nilai suku ke-0 dan ke-1 adalah 0 dan 1, dan nilai suku ke-n selanjutnya adalah hasil penjumlahan dua suku sebelumnya. Secara umum dapat diformulasikan $F_n = F_{n-1} + F_{n-2}$. Berikut ini adalah contoh nilai deret fibonacci hingga suku ke-10. Buatlah program yang mengimplementasikan fungsi rekursif pada deret fibonacci tersebut.

Jawaban :

```
package main
import "fmt"
func fibonacci(n int) int {
    if n <= 1 {
        return n
    }
    return fibonacci(n-1) + fibonacci(n-2)
}

func main() {
    var n int = 10
    fmt.Printf("Deret Fibonacci hingga suku ke-%d:\n", n)
    for i := 0; i <= n; i++ {
        fmt.Printf("%d ", fibonacci(i))
    }
}
```

Output :



```
PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\ASUS USER\109082500119_Samini> go run "c:\Users\ASUS USER\109082500119_Samini\Week 5 - Modul 5\Unguided\Unguid
edi\Unguided1.go"
Deret Fibonacci hingga suku ke-10:
0 1 1 2 3 5 8 13 21 34 55
PS C:\Users\ASUS USER\109082500119_Samini>
```

Penjelasan :

Program ini bekerja dengan menerapkan logika matematika ke dalam fungsi yang memanggil dirinya sendiri, di mana nilai suku ke-n dihitung dengan menjumlahkan hasil dari pemanggilan fungsi untuk suku (n-1) dan (n-2). Proses rekursi ini akan terus bercabang ke bawah hingga mencapai *base case* atau kondisi dasar, yaitu ketika n bernilai 0 atau 1, yang mengembalikan nilai itu sendiri tanpa melakukan perhitungan lebih lanjut.

2. Soal Unguided 2

Buatlah program yang mengimplementasikan rekursif untuk menampilkan faktor bilangan dari suatu N, atau bilangan yang apa saja yang habis membagi N.

Masukan terdiri dari sebuah bilangan bulat positif N

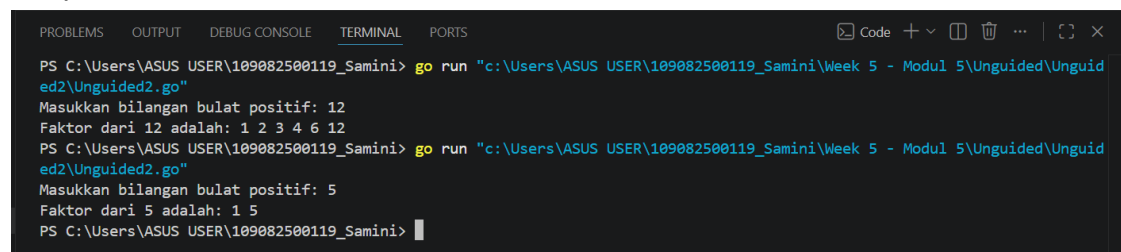
Keluaran terdiri dari barisan bilangan yang menjadi faktor dari N (terurut dari 1 hingga N ya)

Jawaban :

```
package main
import "fmt"
func tampilkanFaktor(n int, i int) {
    if i > n {
        return
    }
    if n%i == 0 {
        fmt.Printf("%d ", i)
    }
    tampilkanFaktor(n, i+1)
}

func main() {
    var n int
    fmt.Print("Masukkan bilangan bulat positif: ")
    fmt.Scan(&n)
    fmt.Printf("Faktor dari %d adalah: ", n)
    tampilkanFaktor(n, 1)
    fmt.Println()
}
```

Output :



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\ASUS USER\109082500119_Samini> go run "c:\Users\ASUS USER\109082500119_Samini\Week 5 - Modul 5\Unguided\Unguid
ed2\Unguided2.go"
Masukkan bilangan bulat positif: 12
Faktor dari 12 adalah: 1 2 3 4 6 12
PS C:\Users\ASUS USER\109082500119_Samini> go run "c:\Users\ASUS USER\109082500119_Samini\Week 5 - Modul 5\Unguided\Unguid
ed2\Unguided2.go"
Masukkan bilangan bulat positif: 5
Faktor dari 5 adalah: 1 5
PS C:\Users\ASUS USER\109082500119_Samini>
```

Penjelasan :

Program ini menggunakan pendekatan rekursi maju dengan fungsi tampilkan Faktor yang menerima dua parameter, yaitu bilangan target N dan variabel pembagi i yang dimulai dari 1. Di dalam fungsi, terdapat kondisi base case yang akan menghentikan rekursi ketika nilai i sudah lebih besar dari N.

3. Soal Unguided 3

Buatlah program yang mengimplementasikan rekursif untuk menampilkan barisan bilangan ganjil.

Masukan terdiri dari sebuah bilangan bulat positif N.

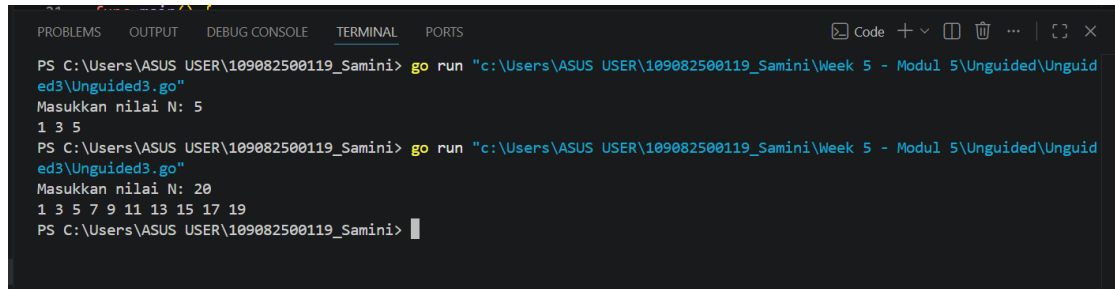
Keluaran terdiri dari barisan bilangan ganjil dari 1 hingga N.

Jawaban :

```
package main
import "fmt"
func cetakGanjil(n int, i int) {
    if i > n {
        return
    }
    if i%2 != 0 {
        fmt.Printf("%d ", i)
    }
    cetakGanjil(n, i+1)
}

func main() {
    var n int
    fmt.Print("Masukkan nilai N: ")
    fmt.Scan(&n)
    cetakGanjil(n, 1)
    fmt.Println()
}
```

Output :



```
PS C:\Users\ASUS USER\109082500119_Samini> go run "c:\Users\ASUS USER\109082500119_Samini\Week 5 - Modul 5\Unguided\Unguided3\Unguided3.go"
Masukkan nilai N: 5
1 3 5
PS C:\Users\ASUS USER\109082500119_Samini> go run "c:\Users\ASUS USER\109082500119_Samini\Week 5 - Modul 5\Unguided\Unguided3\Unguided3.go"
Masukkan nilai N: 20
1 3 5 7 9 11 13 15 17 19
PS C:\Users\ASUS USER\109082500119_Samini>
```

Penjelasan :

Program ini bekerja dengan menjelaskan fungsi rekursif cetak Ganjil yang menerima batas atas N dan pencacah i yang dimulai dari angka 1. Di setiap pemanggilan fungsi, program melakukan pengecekan base case agar rekursi berhenti saat i melewati nilai N, serta menggunakan operator modulus ($i \% 2 \neq 0$) untuk mengidentifikasi dan mencetak angka ganjil sebelum melanjutkan pemanggilan fungsi berikutnya dengan nilai i yang ditambah 1.

D. Kesimpulan

Berdasarkan laporan praktikum di atas, dapat disimpulkan bahwa **rekursif** merupakan teknik pemrograman di mana sebuah subprogram memanggil dirinya sendiri dengan memecah masalah utama menjadi sub-masalah yang lebih kecil dan sejenis. Implementasi rekursif yang efektif sangat bergantung pada penentuan **base-case** sebagai kondisi berhenti untuk mencegah perulangan tak terbatas, serta **recursive-case** untuk menjalankan proses pemanggilan fungsi secara berulang. Melalui berbagai pengujian seperti deret Fibonacci, penentuan faktor bilangan, dan pencetakan barisan ganjil, terlihat bahwa rekursif melibatkan tahap maju hingga kondisi dasar tercapai dan tahap mundur untuk memproses hasil, sehingga teknik ini dapat menjadi alternatif yang efisien dibandingkan metode iteratif untuk menyelesaikan persoalan tertentu.

